

Computer Networks Assignment

Student Name: Samridh Srivastava

UID:24BDA70208

Branch: AIT-CSE

Section/Group: 24AIT KRG-2

Subject Name: Computer Networks

Subject Code: 24CSH-337

Question 1: IP Addressing, Classes, Subnetting & Supernetting

What is an IP Address?

An **IP (Internet Protocol) Address** is a unique 32-bit numerical identifier assigned to every device connected to a network that uses the TCP/IP protocol stack. It operates at the Network Layer (Layer 3) of the OSI model and serves two primary functions:

1. **Host identification:** Uniquely identifying a device on a network.
2. **Logical location and routing:** Allowing network routers to direct traffic across interconnected networks to reach the destination device.

Why Do We Need Classes in IPv4?

In the original **Classful Addressing architecture**, IPv4 addresses were divided into fixed classes (Class A, B, C, D, and E).

Classes were introduced to **scale network allocation** based on organizational size:

- Large organizations (like ISPs or multinational corporations) needed millions of IP addresses per network.
- Medium-sized entities (like universities) needed thousands of host addresses.
- Small businesses needed only a few dozen host addresses.

By dividing the 32-bit address space into distinct classes with predefined boundaries between the **Network ID** and **Host ID**, routers could quickly identify the target network without needing complex variable-length routing masks for every single IP address.

Explanation of IPv4 Classes

An IPv4 address consists of **32 bits**, split into **4 octets** (8 bits each), separated by dots (e.g., 192.168.1.1).

Class Summary Table

|

Class	First Octet Bits (Leading)	First Octet Range	Default Subnet Mask	Network ID Bits	Host ID Bits	Total Networks	Usable Hosts per Network	Primary Purpose
A	0	1 – 127	255.0.0.0	8	24	$2^7 - 2 = 126$	$2^{24} - 2 = 16,777,214$	Extremely large networks
B	10	128 – 191	255.255.0.0	16	16	$2^{14} = 16,384$	$2^{16} - 2 = 65,534$	Medium-to-large networks
C	110	192 – 223	255.255.255.0	24	8	$2^{21} = 2,097,152$	$2^8 - 2 = 254$	Small Local Area Networks (LANs)
D	1110	224 – 239	N/A	N/A	N/A	N/A	N/A	Multicasting
E	1111	240 – 255	N/A	N/A	N/A	N/A	N/A	Reserved for Research & Future Use

Important Notes on Host Calculations:

- In every network, **2 addresses are reserved**:
 - Network Address**: The first address (Host bits all 0s), used to identify the network itself.
 - Broadcast Address**: The last address (Host bits all 1s), used to send data to all hosts on the network.
- In Class A, 127.0.0.0 to 127.255.255.255 is reserved for **loopback testing** (diagnostics), leaving 126 usable Class A networks.

Subnetting and Supernetting

1. Subnetting

Subnetting is the process of dividing a single large physical network into multiple smaller logical networks (subnets) by **borrowing bits from the Host ID portion** to add to the Network ID.

- Purpose**: Reduces network congestion (smaller broadcast domains), enhances security, and improves IP management.
- Example**:
 - Take a Class C address block: 192.168.1.0/24 (Default mask: 255.255.255.0).
 - Total available host bits = 8 ($2^8 - 2 = 254$ usable hosts).
 - Suppose we borrow **2 bits** from the host portion for subnetting:
 - New Subnet Mask bits = $24 + 2 = 26$ bits (/26 or 255.255.255.192).

- Number of subnets created = $2^2 = 4$ subnets.
- Remaining host bits = $8 - 2 = 6$ bits.
- Usable hosts per subnet = $2^6 - 2 = 62$ hosts.
- **Resulting Subnets:**
 1. Subnet 1: 192.168.1.0/26 (Usable IP range: 192.168.1.1 to 192.168.1.62)
 2. Subnet 2: 192.168.1.64/26 (Usable IP range: 192.168.1.65 to 192.168.1.126)
 3. Subnet 3: 192.168.1.128/26 (Usable IP range: 192.168.1.129 to 192.168.1.190)
 4. Subnet 4: 192.168.1.192/26 (Usable IP range: 192.168.1.193 to 192.168.1.254)

2. Supernetting

Supernetting (also known as Classless Inter-Domain Routing / CIDR aggregation) is the inverse of subnetting. It combines multiple contiguous smaller networks into a single larger network by **borrowing bits from the Network ID portion**.

- **Purpose:** Reduces the size of routing tables on core Internet routers by summarizing multiple routes into a single route entry.
- **Example:**
 - An organization owns four contiguous Class C networks:
 1. 192.168.0.0/24
 2. 192.168.1.0/24
 3. 192.168.2.0/24
 4. 192.168.3.0/24
 - Convert the 3rd octets to binary:
 - $0 = 00000000_2$
 - $1 = 00000001_2$
 - $2 = 00000010_2$
 - $3 = 00000011_2$
 - The first **22 bits** across all four networks are identical (**11000000.10101000.000000**).
 - **Supernet Summary Address:** 192.168.0.0/22 with Supernet Mask 255.255.252.0.
 -

Question 2: Link State vs. Distance Vector Routing

Comparison Table

Parameter	Distance Vector Routing	Link State Routing
-----------	-------------------------	--------------------

Algorithm Used	Bellman-Ford Algorithm	Dijkstra's Shortest Path First (SPF) Algorithm
----------------	------------------------	--

| **Topology Knowledge** | Knows only immediate physical neighbors and their reported distances ("Routing by rumor"). | Has complete knowledge of the entire network topology graph. |

| **Update Mechanism** | Sends periodic updates of its **entire routing table** to immediate neighbors only. | Floods **Link State Advertisements (LSAs)** containing local link status to **all routers** in the network when changes occur. |

| **Convergence Speed** | **Slow** (Susceptible to routing loops and Count-to-Infinity problems). | **Fast** (Rapid reaction to link failures). |

| **Resource Overhead** | Low CPU and RAM requirements; higher bandwidth usage due to periodic updates. | High CPU and memory consumption to store network graph and calculate SPF trees. |

| **Example Protocols** | RIP (Routing Information Protocol), EIGRP (hybrid). | OSPF (Open Shortest Path First), IS-IS. |

How Algorithms Identify Least Cost Path (With Examples)

1. Distance Vector Routing (Bellman-Ford Algorithm)

Each router maintains a distance vector table consisting of (Destination, Cost, Next Hop). Routers exchange vector tables with immediate neighbors and iteratively compute the minimum path cost using the Bellman-Ford equation:

$$D_X(Y) = \min_V \{c(X, V) + D_V(Y)\}$$

Where $c(X, V)$ is the direct link cost from router X to neighbor V , and $D_V(Y)$ is neighbor V 's reported cost to destination Y .

- **Step-by-Step Example:**

- Consider three connected routers: A , B , and C .
- Direct Link Costs: $c(A, B) = 2$, $c(A, C) = 7$, $c(B, C) = 1$.
- **Initial State at Router A :**
 - Distance to $B = 2$ (direct)
 - Distance to $C = 7$ (direct)
- **Update Phase:**
 - Neighbor B advertises its routing table to A . B reports its distance to C is

- **Calculation at Router A for destination C :**

$$D_A(C) = \min\{c(A, C), c(A, B) + D_B(C)\}$$

$$D_A(C) = \min\{7, 2 + 1\} = \min\{7, 3\} = 3$$

- **Outcome:** Router A updates its table: Least cost to reach C is **3**, via Next Hop B .

2. Link State Routing (Dijkstra's Algorithm)

In Link State routing, every router receives Link State Advertisements (LSAs) from all nodes, creating a complete view of the network as a graph $G = (V, E)$. Router S then runs Dijkstra's algorithm to compute the shortest path tree from S to all other destinations.

- **Dijkstra's Algorithm Notation:**

- $D(v)$: Current cost of the least-cost path from source S to destination v .
- $p(v)$: Predecessor node along path to v .
- N' : Set of nodes whose least-cost path is definitively known.

- **Step-by-Step Example:**

- Network topology with Source Node = A :
 - Links: $A - B = 2$, $A - C = 5$, $B - C = 1$, $B - D = 4$, $C - D = 1$.
- **Algorithm Steps:**
 - 1. Initialization:**
 - $N' = \{A\}$
 - $D(B) = 2, p(B) = A$
 - $D(C) = 5, p(C) = A$
 - $D(D) = \infty$
 - 2. Iteration 1:**
 - Find minimum $D(w)$ outside N' : Select B ($D(B) = 2$).
 - Add B to N' : $N' = \{A, B\}$.

- Update neighbors of B :
 - $D(C) = \min(5, D(B) + c(B, C)) = \min(5, 2 + 1) = 3$
(Predecessor becomes B).
 - $D(D) = \min(\infty, D(B) + c(B, D)) = \min(\infty, 2 + 4) = 6$
(Predecessor becomes B).
- 3. **Iteration 2:**
 - Find minimum $D(w)$ outside N' : Select C ($D(C) = 3$).
 - Add C to N' : $N' = \{A, B, C\}$.
 - Update neighbors of C :
 - $D(D) = \min(6, D(C) + c(C, D)) = \min(6, 3 + 1) = 4$
(Predecessor becomes C).
- 4. **Iteration 3:**
 - Select D ($D(D) = 4$). Add D to N' : $N' = \{A, B, C, D\}$. Algorithm terminates.
- **Outcome:**
 - Shortest path to C : $A \rightarrow B \rightarrow C$ (Cost = 3).
 - Shortest path to D : $A \rightarrow B \rightarrow C \rightarrow D$ (Cost = 4).

Question 3: Congestion Control, Traffic Shaping & Fragmentation

What is Network Congestion & Why Control It?

Network Congestion occurs when the rate of incoming data packets entering a network segment exceeds the processing capacity or bandwidth limit of intermediate routers and links. When buffer capacities fill up, routers begin discarding excess packets.

Reasons We Need Congestion Control:

1. **Prevent Packet Loss & Buffer Overflow:** Unmanaged queues cause packet drops, forcing transport protocols (like TCP) to retransmit lost data.
2. **Avoid Congestion Collapse:** Uncontrolled retransmissions consume remaining network capacity, leading to a state where useful network throughput drops near zero.
3. **Minimize Latency and Jitter:** Excessive queuing delays at intermediate routers increase

round-trip times (RTT) and degrade real-time traffic (such as VoIP or video calls).

Traffic Shaping: Leaky Bucket vs. Token Bucket

Both algorithms are used for **traffic shaping** (controlling the rate of transmitted data), but they enforce traffic flow differently.

Comparison Table

| Feature | Leaky Bucket Algorithm | Token Bucket Algorithm |

| Primary Goal | Flattens bursty traffic into a **strictly continuous, uniform output rate**. | Regulates average transmission rate while **allowing controlled bursty traffic**. |

| Burst Handling | Idle periods do **not** grant future burst capacity; packets arriving above the leak rate are queued or dropped. | Idle periods allow tokens to accumulate, enabling subsequent packet bursts up to bucket capacity. |

| Output Rate | Fixed/Constant rate. | Variable rate (constant rate for steady streams, immediate high rate for bursts). |

| Overflow Action | When the bucket fills up, excess **packets are discarded**. | When the bucket fills up, excess **tokens are discarded** (packets are not lost). |

Numerical Examples

- **Leaky Bucket Example:**

- Bucket Capacity = 10 MB .
- Fixed Output (Leak) Rate = 2 MB/s .
- **Scenario:** A burst of 8 MB data arrives at $t = 0$:
 1. The 8 MB fits into the 10 MB bucket capacity.
 2. The bucket releases data at a constant rate of 2 MB/s .
 3. It takes exactly $\frac{8 \text{ MB}}{2 \text{ MB/s}} = 4 \text{ seconds}$ to process the burst smoothly.

- **Token Bucket Example:**

- Bucket Capacity = 5 tokens .
- Token Generation Rate = 1 token/sec (where 1 token allows sending a 1 KB packet).
- **Scenario:**
 1. System is idle for 5 seconds : Bucket fills to max capacity (5 tokens).
 2. At $t = 5 \text{ seconds}$, a sudden burst of 5 KB data (5 packets) arrives.

3. The system consumes all **5 tokens** instantly and transmits all **5 KB** of data **immediately in a single burst**.

Why Do We Need Fragmentation?

Fragmentation is the network-layer process of dividing a large IP datagram into smaller pieces (fragments) so that they can be transmitted across a physical link whose **Maximum Transmission Unit (MTU)** is smaller than the packet size.

Reasons for Fragmentation:

1. **Diverse Link Technologies:** Networks consist of heterogeneous physical mediums with differing MTU limits (e.g., standard Ethernet MTU is **1500 bytes** , while PPP links or older WAN standards may have MTUs of **576 bytes**).
2. **Interoperability:** Intermediate routers along a packet's path must be able to forward packets without dropping them simply because a downstream link supports smaller frame sizes.

IP Header Fields Supporting Fragmentation:

- **Identification (16 bits):** An integer assigned by the sender to uniquely identify all fragments originating from the same original packet.
- **Flags (3 bits):**
 - **DF (Don't Fragment):** If set to 1, routers are prohibited from fragmenting the packet (and will drop it with an ICMP error if MTU is exceeded).
 - **MF (More Fragments):** Set to 1 for all fragments except the last fragment (which has MF = 0).
- **Fragment Offset (13 bits):** Indicates the relative position of the fragment's payload with respect to the start of the original unfragmented payload, measured in **8-byte (64-bit) blocks**.